



Ejercicios de ramificación y poda

Para cada ejercicio propuesto a continuación:

- a) Define una representación para las soluciones y describe el árbol del espacio de soluciones.
- b) Define la función de coste $c(x)$ que hay que minimizar (donde x es un nodo del árbol de búsqueda).
- c) Define una función de coste estimado $\hat{c}(x)$ que se utilizará para la ramificación.
- d) Define una función de poda $\mathcal{U}(x)$.

Ej. 1 — Para garantizar la seguridad del campus Río Ebro el Rector quiere instalar cámaras que vigilen todas sus calles. Una calle está vigilada por una cámara en uno de sus extremos (*vértices*) y cada cámara puede vigilar todas las calles que convergen en un vértice hasta el vértice siguiente. Por ejemplo, consideramos las siguientes calles: $(1, 2), (1, 3), (1, 4), (2, 5), (4, 6)$ y $(6, 7)$. Una posible solución es colocar las cámaras en los vértices $\{2, 3, 4, 5, 7\}$ y una solución óptima es: $\{1, 5, 6\}$. Diseña un algoritmo para encontrar el mínimo número de cámaras necesario para vigilar todo el campus.

Ej. 2 — Has invitado a tus amigos a casa y están hambrientos. No tienes nada que ofrecerles de comer, así que decides ordenar una pizza de *Enzo* que hace pizzas enormes, con la posibilidad de pedir más de un ingrediente en una pizza. Tienes la lista de los m ingredientes que es posible pedir (con anchoas, queso, setas, chorizo, etc.) y cada amigo te ha dicho qué ingredientes prefiere.

En base a las preferencias, has rellenado una matriz $n \times m$, donde cada elemento p_{ij} de la matriz es igual a 1 si al amigo i le gusta el ingrediente j ; si no, es 0 (cero). Diseña un algoritmo que encuentre un conjunto de ingredientes $T \subseteq \{t_1, \dots, t_m\}$ que sea el más pequeño posible para comprar una única pizza y a la vez satisfacer a todos tus n amigos, es decir para cada amigo $i \in \{1, \dots, n\} : \sum_{j \in T} p_{ij} \geq 1$.



Ej. 3 — Se desea construir aeropuertos en un país con muchas ciudades. Cada aeropuerto debe estar situado en una ciudad y todas las ciudades deben tener un aeropuerto a menos de 100 km. Como datos de entrada tenemos: el conjunto total, $T = \{1, 2, \dots, n\}$, de ciudades; y para cada ciudad $i (i = 1, \dots, n)$, conocemos el subconjunto de ciudades, $S_i \subseteq T$, que están a menos de 100 km de i (ella misma incluida). Diseña un algoritmo para calcular el mínimo número de aeropuertos necesarios: este número es el mínimo número de subconjuntos S_i que “cubren” todas las ciudades (es decir, cuya unión es T).

Ej. 4 — Dadas n personas y n tareas, sea $B[i, j]$ el beneficio asociado a que la persona i se encargue de realizar la tarea j . Diseña un algoritmo para encontrar la asignación de personas a tareas que maximice el beneficio obtenido. Para ello, considera plantear el problema como *problema de mínimo* y aplicar el esquema de ramificación y poda de mínimo coste.

Ej. 5 — Para celebrar el fin de curso, tú y tus compañeros de Algoritmia Básica habéis organizado una quedada en el parque de la Expo. Has pensado que podría ser divertido jugar a un “tira y afloja” después de la comida y bebida. Para el tira y afloja, tenéis que dividirlos en dos equipos tan equilibrados como sea posible; en particular, el número de personas en los dos equipos no debe diferir en más de uno y el peso total de las personas en cada equipo debe ser lo más igual posible. Considera que sois n participantes y conoces el peso p_i de cada participante i ($i = 1, \dots, n$). Diseña un algoritmo para formar los dos equipos considerando las mencionadas restricciones.

Ej. 6 — Un restaurante está preparando un banquete para n personas. El cocinero es capaz de cocinar q menús diferentes $\{m_1, \dots, m_q\}$, pero conoce los gustos de cada persona y quiere minimizar el número de menús diferentes a cocinar a la vez que agradar a las n personas. Interesa por tanto obtener el conjunto más pequeño de menús que puede satisfacer a toda la gente. Cada menú m_i se puede expresar como un vector de ceros y unos, es decir $m_i \in \{0, 1\}^n$, tal que la componente j de m_i , m_{ij} , es igual a 1 si y sólo si a la persona j le gusta el menú i . Diseña un algoritmo para obtener el mínimo conjunto Z que satisface:

$$\sum_{m_i \in Z} m_i \geq \mathbf{1}$$

donde $\mathbf{1}$ denota a un vector de dimensión n cuyas componentes son todas iguales a 1.

Ej. 7 — Para sensibilizar los niños sobre la protección de los animales, en un colegio se ha decidido que cada niño apadrine un animal mediante una adopción simbólica de una especie diferente. Hay n niños en el colegio: cada niño ha dado su preferencia para cada una de las n especies diferentes que se pretenden apadrinar (es decir, 0 no me gusta, 1 me gusta). Diseña un algoritmo para encontrar una asignación niño-animal que maximice la satisfacción global. La satisfacción global se calcula sumando las preferencias dadas por los niños por apadrinar una especie. Para ello, puedes plantear el problema como *problema de mínimo* y aplicar el esquema de ramificación y poda de **mínimo coste**.

Ej. 8 — El diseño de la página es muy importante para los periódicos y revistas. Los artículos se colocan en diferentes posiciones para maximizar el impacto visual. Cada artículo tiene un tamaño determinado y una ubicación preferencial fija. Cada artículo siempre cabe en una página, sin embargo no se pueden incluir todos en una sola página (tampoco pueden solaparse).

Representamos la página con un rectángulo, definido por las esquinas superior izquierda $(0, 0)$ y inferior derecha (W, L) . Sean $a_1, \dots, a_n$ los artículos a publicar: cada artículo a_i tiene que colocarse en un rectángulo (w_i, h_i, x_i, y_i) — donde w_i es la anchura, h_i la altura, y x_i, y_i las coordenadas cartesianas de la posición de la esquina superior izquierda. Se quiere diseñar un algoritmo para encontrar el conjunto de artículos a colocar en la página que minimice la cantidad de espacio libre en la página (es decir, espacio NO ocupado por los artículos). Dos artículos se solapan si la intersección de los dos rectángulos correspondientes no es vacía: los artículos pueden compartir un lado, como por ejemplo los artículos A y B en la figura, o una esquina.

